Trainer Guide: Spring Boot Session 3 - Hands-on Lab

A walk-through script for springboot-session3-lab.html - building the layered Post API with a DTO and custom exceptions. Explain every step out loud.

Presenter: Purvam Prajapati - Faculty, AHD
Companion to: springboot-session3-lab.html

Each lab has an ON SCREEN reminder and a SAY / WALK THROUGH section you can read out as you lead. EMPHASISE and DELIVERY TIP notes are for you only.

Setup

Lab flow & build order

ON SCREEN

The lab page with a sticky progress bar, an 8-step checklist, and colour-coded lab sections with copy buttons.

SAY / WALK THROUGH

Today we build a properly layered Post API, one file at a time, in a deliberate order: entity, repository, DTO, custom exception, service, then controller, then we test the whole thing. The progress bar tracks eight milestones. The order matters - each file depends on the ones before it, so we build bottom-up: the data pieces first, then the logic, then the web layer on top. By the end you'll have a clean, persistent, professional API that returns proper error messages.

Pre-requisites & application.properties

ON SCREEN

Pre-requisites - a fresh project and Postman - then the full application.properties block for H2, explained line by line.

SAY / WALK THROUGH

Pre-requisites: a fresh Spring Boot project with four dependencies - Web, Spring Data JPA, H2, and DevTools. Then fill in application dot properties. The block sets the H2 in-memory database URL, enables the H2 console at slash h2-console, turns on show-sql so you can watch the SQL Hibernate runs, and sets ddl-auto to update so Hibernate creates the table from your entity automatically. Each line has a short explanation on the sheet.

DELIVERY TIP

Remind everyone to generate the getters and setters for the entity and the DTO the moment they create each class - a missing getter is the single biggest cause of empty JSON responses later in this lab.

The labs - build the layered API

Lab 1 - Create the Post Entity

ON SCREEN

Steps to create an entity package and Post.java with @Entity, @Table, @Id, @GeneratedValue, four fields, and their getters and setters.

SAY / WALK THROUGH

Lab one - the entity. Create an entity package and a Post class. Annotate it with at-Entity to mark it as a database table and at-Table named posts. Inside, at-Id and at-GeneratedValue on a Long id, then three String fields: title, description, content. Two things to call out. First, add getters and setters for all four fields - let the IDE generate them, with Alt and Insert in IntelliJ or right-click Source in Eclipse; JPA and Jackson both rely on them. And second, in Spring Boot 3 the imports come from jakarta dot persistence, not javax - flag that, because copied-from-the-internet code often uses the old javax and won't compile.

Lab 2 - Create the Repository

ON SCREEN

Steps to create a repository package and PostRepository interface extending JpaRepository<Post, Long>.

SAY / WALK THROUGH

Lab two is the shortest in the whole series. Create a repository package and an interface, PostRepository, that extends JpaRepository of Post and Long - the entity type and the id type. Add at-Repository. And... that's it. This single empty interface gives you save, findById, findAll, deleteById, count, and more - with zero implementation written by you. Spring Data generates the implementation at runtime. This is the payoff of all that JDBC we'd otherwise write by hand: the repository layer is essentially free.

Lab 3 - Create the PostDTO

ON SCREEN

Steps to create a dto package and PostDTO.java with the same four fields, their getters and setters, and no JPA annotations.

SAY / WALK THROUGH

Lab three - the DTO. Create a dto package and a PostDTO class with the same four fields as the entity - id, title, description, content - plus their getters and setters. But notice what's missing: no at-Entity, no at-Id, none of the JPA annotations. That's the whole point. The DTO and the entity look almost identical right now, and that's fine. The entity maps to the database and may grow internal fields later; the DTO is the public face that only ever exposes what the API should return. Keeping them separate means changing the table never accidentally changes your API contract.

Lab 4 - Create the Custom Exception

ON SCREEN

Steps to create an exception package and ResourceNotFoundException extending RuntimeException with @ResponseStatus(NOT_FOUND).

SAY / WALK THROUGH

Lab four - the custom exception that gives clients clean errors. Create an exception package and a class, ResourceNotFoundException, that extends RuntimeException. Give it a constructor taking a String message and passing it to super. Then, above the class, the key annotation:

at-ResponseStatus of HttpStatus dot NOT underscore FOUND. That annotation is the magic. It tells Spring: whenever this exception is thrown and not caught, set the HTTP response to 404 automatically. No try-catch, no extra configuration - just the annotation. We'll throw this in the service in the next lab.

Lab 5 - Create the Service Layer

ON SCREEN

Steps to create the PostService interface and PostServiceImpl with @Service, @Autowired repository, two mapper methods, and all five methods using orElseThrow.

SAY / WALK THROUGH

Lab five is the biggest block and the heart of the session. First, a PostService interface with five method signatures - create, getAll, getById, update, delete. Coding to an interface is good practice - easy to swap or test. Then PostServiceImpl, annotated at-Service, with the repository injected via at-Autowired. Read the two private helper methods first, because they're the core of the DTO pattern: mapToDTO copies an entity into a DTO on the way out, and mapToEntity copies a DTO into an entity on the way in. Then the five methods use them - createPost maps DTO to entity, saves, maps back. And getById and updatePost use findById dot orElseThrow, throwing our ResourceNotFoundException when the id is missing.

EMPHASISE

Slow down on orElseThrow and read it like English: 'find by id, or else throw this exception.' findById returns an Optional; orElseThrow gives you the Post if present, or throws if empty - no null checks, and the client gets the clean 404 message we wrote.

Lab 6 - Create the Controller

ON SCREEN

Steps to create PostController with @RestController, @RequestMapping('/api/posts'), injected service, and five endpoints using ResponseEntity.

SAY / WALK THROUGH

Lab six - the web layer. Create PostController with at-RestController and at-RequestMapping slash api slash posts at the class level, and inject the PostService. Then five endpoints. POST returns a ResponseEntity with status 201 Created. GET-all returns the list of DTOs. GET-by-id and PUT return ResponseEntity dot ok, and they get a 404 automatically when the service throws. DELETE returns a 200 with a confirmation message. Notice the controller is thin - it just translates HTTP to service calls and back. No business logic, no SQL. Because at-RequestMapping is on the class, each method only needs its suffix - the full path is assembled for you, like slash api slash posts slash id.

Lab 7 - Test Everything (H2 + Postman)

ON SCREEN

Four test parts: verify the POSTS table in the H2 console, POST a record (201), GET valid vs invalid id (200 vs 404), and PUT an update.

SAY / WALK THROUGH

Lab seven - we prove it all works, in four parts. Part A: start the app, open slash h2-console, connect with the JDBC URL jdbc colon h2 colon mem colon testdb, and confirm Hibernate auto-created the POSTS table. Part B: in Postman, POST a new post as JSON - expect 201 Created with a generated id, then run SELECT star FROM POSTS in the console to see the row. Part C - the money moment: GET slash api slash posts slash 1 returns 200 with the post, and GET slash api slash posts slash 99 returns a clean 404 with the message 'Post not found with id equals 99'. Part D: PUT an update to post 1 and confirm the title changed but the id stayed the same.

EMPHASISE

Part C is the highlight of the whole session - make everyone do the invalid-id GET and read the 404 message aloud. That clean 404 is your custom exception and at-ResponseStatus working together; it's the visible payoff of the exception-handling theory.

Lab 8 - Bonus: Spring Security (observe only)

ON SCREEN

A bonus, observe-only lab: add the security starter, restart, see the auto login page, find the generated password, then set custom credentials.

SAY / WALK THROUGH

Lab eight is observe-only - you drive, they watch. Add the spring-boot-starter-security dependency and restart. Now navigate to any endpoint in the browser and a login page appears automatically - you wrote no security code. Check the console for the line 'Using generated security password' and log in as user with that password. Then add two lines to application dot properties - a custom username and password - restart, and log in with those instead. The point to make: this is form-based security, a starting point. Production APIs use token-based security with JWT and roles, which is a dedicated later session. If you want to keep testing freely afterwards, remove the dependency again.

Reference & close

Complete files reference & wrap-up

ON SCREEN

The 'Complete Files Reference' with all five Java files, plus the quick-reference card.

SAY / WALK THROUGH

The catch-up safety net at the bottom has all five Java files - Post, PostDTO, the exception, the service interface and impl, and the controller - ready to paste in build order. There's also a quick-reference card covering DTO, getters and setters, at-ResponseStatus, orElseThrow, and ResponseEntity. To wrap: today you built a clean, layered, persistent API. A request flows from the controller, through the service that maps DTO to entity, to the repository and the H2 database - and missing records come back as clean 404s instead of stack traces. This is the fundamental unit of a real system. In Session 4 we run several services like this together, behind a gateway. Great work.